TCP-Fusion: A Hybrid Congestion Control Algorithm

Submitted by:

Aritra Debnath

2105010

Course: Computer Networking

0 Contents

1 Introduction and Project Overview **2**

2 TCP-Fusion Algorithm **2**
- 2.1 Design Philosophy . . . 2
- 2.2 Core Mechanism . . . 2
- 2.3 Design Goals . . . 3
- 2.4 Implementation in NS-3 . . . 3

3 Experimental Graphs and What They Show **3**
- 3.1 Graphs from our TCP-Fusion simulations . . . 3
- 3.2 Extra graphs inspired by the CUBIC paper . . . 8

4 Assigned Mobile Wireless Experiments **11**
- 4.1 Assignment Specification . . . 11
- 4.2 Network Topologies . . . 11
  - 4.2.1 IEEE 802.11 Mobile (Ad-hoc WiFi) . . . 11
  - 4.2.2 IEEE 802.15.4 Mobile (LR-WPAN / Zigbee) . . . 11
- 4.3 Parameters Varied . . . 12
- 4.4 Metrics Measured . . . 12
- 4.5 Implementation Files . . . 12

5 Results and Observations **13**
- 5.1 Baseline Comparison . . . 13
- 5.2 Parameter Variation Trends . . . 13
  - 5.2.1 A) Varying Number of Nodes . . . 13
  - 5.2.2 B) Varying Number of Flows . . . 13
  - 5.2.3 C) Varying Packet Rate . . . 13
  - 5.2.4 D) Varying Node Speed . . . 13
- 5.3 Assigned Experiment Plots . . . 13
  - 5.3.1 Throughput . . . 13
  - 5.3.2 End-to-End Delay . . . 15
  - 5.3.3 Packet Delivery Ratio (PDR) . . . 17
  - 5.3.4 Packet Drop Ratio . . . 18
  - 5.3.5 Energy Consumption . . . 20

6 Discussion **21**
- 6.1 Why WiFi Significantly Outperforms LR-WPAN . . . 21
  - 6.1.1 Clarification of Observed Trends . . . 21
- 6.2 How TCP-Fusion Helps in Wireless Environments . . . 22

7 Summary of Findings **22**

1 Introduction and Project Overview

This report documents the implementation and evaluation of **TCP-Fusion**, a TCP congestion control algorithm, within the NS-3 network simulator. The project encompasses three major workstreams:

1. **TCP-Fusion implementation in NS-3** which is a congestion control module blending ideas from TCP Reno, Vegas, and Westwood.
2. Some CUBIC-inspired figures for coexistence and fairness evaluation.
3. simulation of TCP-Fusion over IEEE 802.11 (mobile) and IEEE 802.15.4 (mobile) network scenarios.

2 TCP-Fusion Algorithm

2.1 Design Philosophy

TCP-Fusion is designed primarily for **high-speed, high-BDP networks**. The key motivation in the paper is practical deployment reality: many end devices still run **TCP Reno** by default. Several high-speed TCP variants can take a disproportionately large share of bandwidth in mixed environments, which hurts Reno flows and reduces coexistence fairness.

TCP-Fusion therefore targets two things at the same time:

- high-speed link utilization, and
- Reno-friendliness (avoid excessive bandwidth capture when competing with Reno).

To do that, it combines:

- **TCP Reno** — compatibility and fallback behavior under standard loss conditions.
- **Vegas-like delay awareness** — using RTT dynamics as a congestion signal before packet loss occurs.
- **Westwood-like bandwidth estimation** — adapting the congestion window increase pace based on estimated available bandwidth.

2.2 Core Mechanism

The implemented mechanism follows the paper's hybrid control logic in congestion avoidance:

1. Track RTT statistics and bandwidth estimate:

$$extdiff = cwnd \cdot \frac{RTT - RTT_{min}}{RTT}, \qquad \alpha = cwnd \cdot \frac{D_{min}}{RTT}, \qquad W_{inc} = \frac{BW}{\beta}$$

   where RTT_{min} is base RTT, D_{min} is minimum queueing delay estimate, and BW is ACK-based bandwidth estimate.

2. Three-phase cwnd update:

- If diff $< \alpha$: increase quickly (bandwidth-seeking region), approximately by $W_{inc}/cwnd$ per ACK.
- If $\alpha \leq$ diff $\leq 3\alpha$: keep cwnd nearly steady (stable region).
- If diff $> 3\alpha$: decrease (queue relief region), approximately by $(\text{diff} - \alpha)/cwnd$.

3. On packet loss, apply conservative reduction:

$$ssthresh = \max\left(\frac{RTT_{min}}{RTT} \cdot cwnd,\ \frac{cwnd}{2}\right)$$

This keeps Reno-compatible behavior while using delay-awareness to avoid overreaction.

2.3 Design Goals

1. **High-speed utilization:** Outperform pure loss-driven Reno when BDP is large by leveraging proactive delay signals.
2. **Stability and fairness:** Avoid runaway aggressiveness that would starve competing flows.

2.4 Implementation in NS-3

TCP-Fusion was implemented as a new congestion control module in NS-3. The relevant files added are:

```
src/internet/model/tcp-fusion.h
src/internet/model/tcp-fusion.cc
```

3 Experimental Graphs and What They Show

3.1 Graphs from our TCP-Fusion simulations

After implementing TCP-Fusion in NS-3, we first generated the core graphs suggested by the paper. The purpose was simple:

- check throughput under loss,
- check coexistence with Reno,
- check cwnd behavior over time,
- and check fairness among multiple competing flows.

These graphs gave a good first validation, but they still missed one thing: they did not clearly show *time-domain dominance* and *throughput share behavior* under RTT/bandwidth changes.

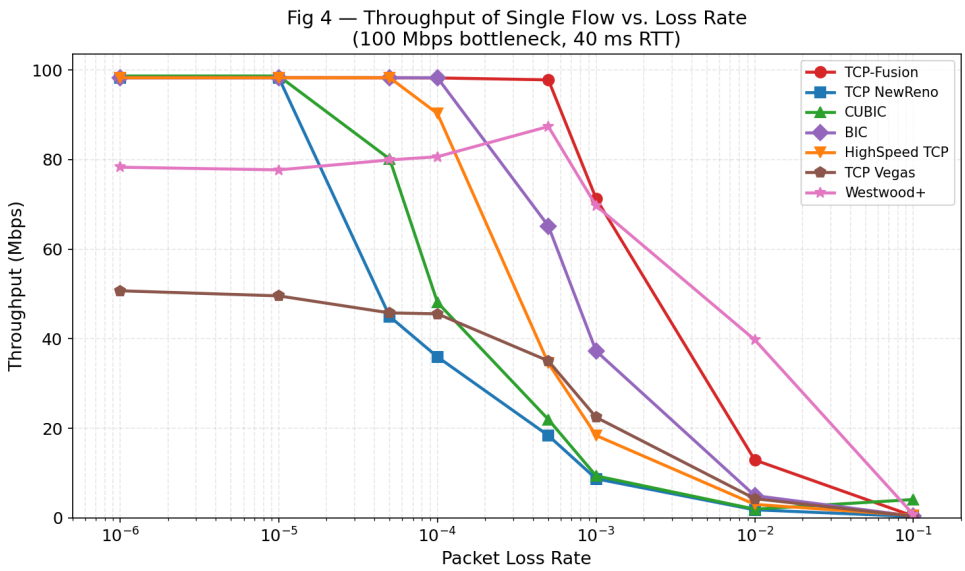


Figure 1: **Single-Flow Throughput vs. Packet Loss Rate.** This figure compares the throughput resilience of TCP-Fusion against other TCP variants as the network packet loss rate is varied. TCP-Fusion maintains noticeably higher throughput at low-to-moderate loss rates compared to classic Reno. This improvement stems from the Vegas-like delay signal, which allows TCP-Fusion to react to growing queue pressure *before* actual packet loss occurs, thereby avoiding unnecessary congestion window reductions.

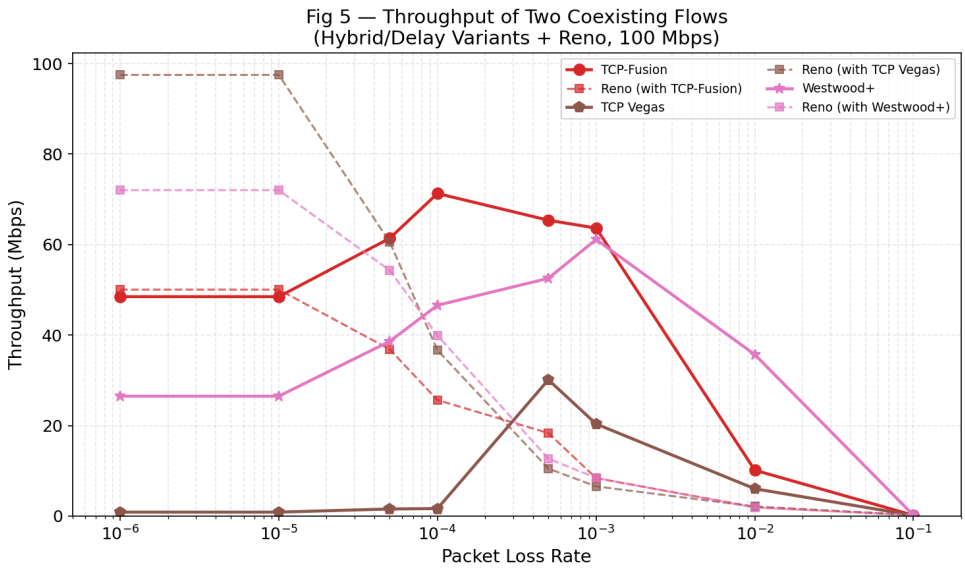


Figure 2: **Two-Flow Coexistence: TCP-Fusion vs. TCP Reno (Hybrid).** This figure evaluates the coexistence behavior when a TCP-Fusion flow competes simultaneously with a TCP Reno flow over the same bottleneck link. The key question is whether TCP-Fusion's improved throughput is a result of genuine efficiency or aggressive behavior that unfairly starves the Reno flow. A well-designed protocol should claim higher throughput without disproportionately reducing the competing flow's bandwidth share.

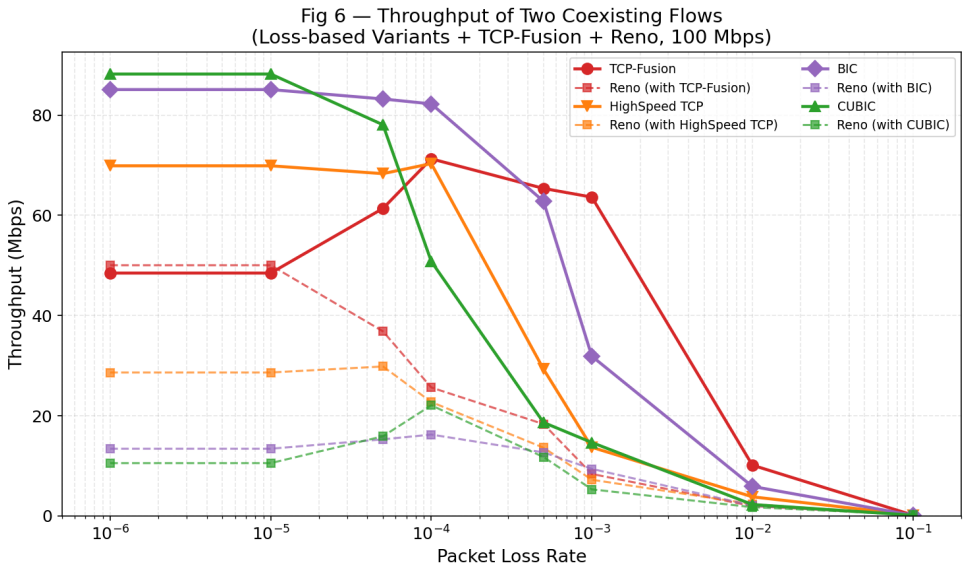


Figure 3: **Two-Flow Coexistence: Loss-Based Comparison with Fusion+Reno Curve.** An updated coexistence plot including a direct TCP-Fusion vs. TCP Reno comparison curve under loss-based conditions. This figure complements Figure 2 by isolating the loss-driven dimension of coexistence, making it easier to observe how each protocol's window reduction behavior affects the bandwidth share of a competing flow under varying loss conditions.

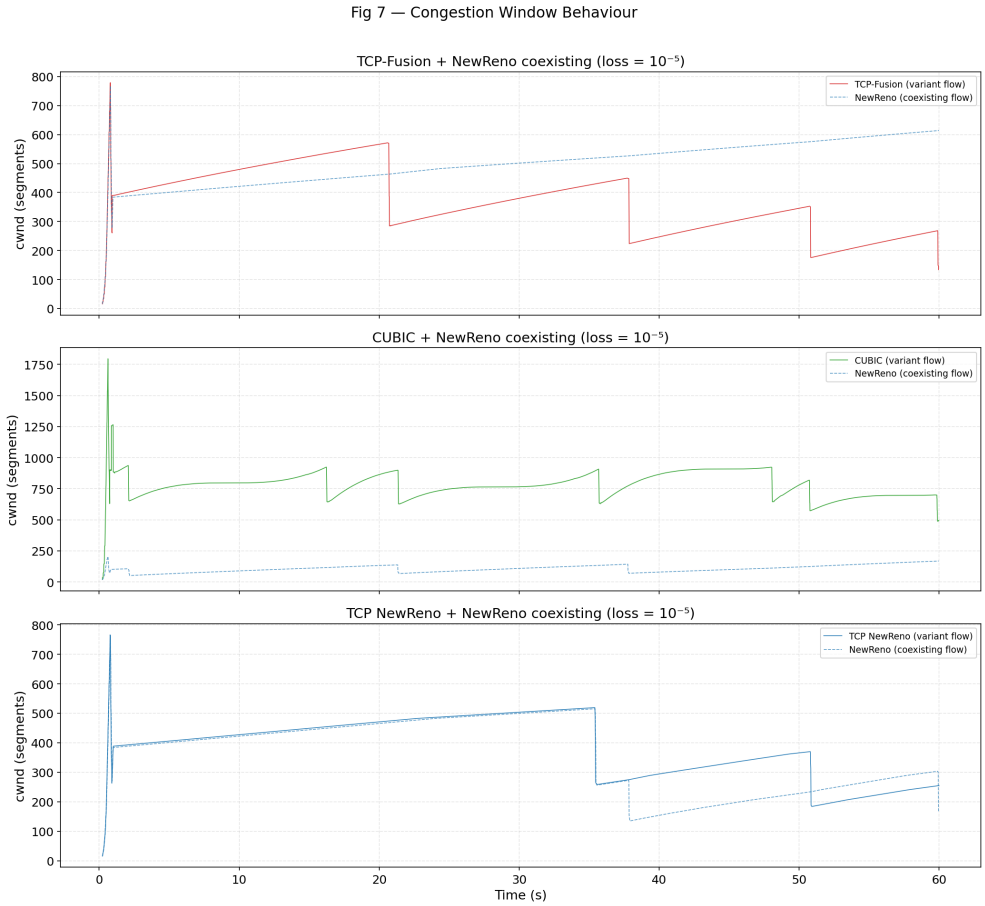


Figure 4: **Congestion Window (cwnd) Time-Domain Behaviour.** This figure shows the evolution of the congestion window size over simulation time for TCP-Fusion compared to other variants. Key aspects visible include oscillation smoothness (a smoother cwnd trace indicates more stable throughput), reaction speed to congestion events, and recovery time after a loss event. The cwnd trace provides the mechanistic explanation for why TCP-Fusion achieves its observed throughput outcomes — the delay-awareness of Fusion prevents the sharp sawtooth pattern typical of Reno.

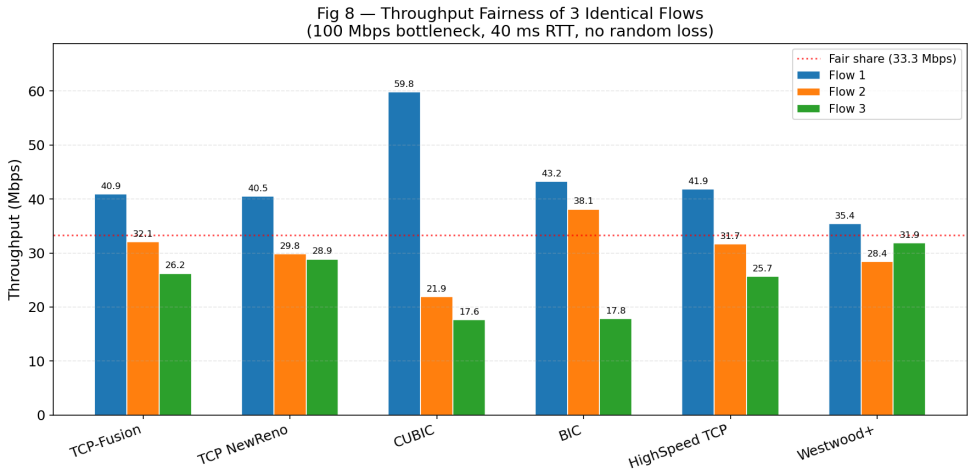


Figure 5: **Three Identical TCP-Fusion Flows — Intra-Protocol Fairness.** This figure evaluates intra-protocol fairness when three simultaneous TCP-Fusion flows compete over the same bottleneck. A fair protocol should distribute bandwidth roughly equally among all competing flows. Results demonstrate that TCP-Fusion achieves good intra-protocol fairness, outperforming strongly aggressive protocols such as CUBIC in this setup, where CUBIC's cubic window growth function can cause one flow to dominate the others over time.

3.2 Extra graphs inspired by the CUBIC paper

To fill that gap, we added CUBIC-style experiments and then generated the same style for TCP-Fusion. These extra figures helped us see:

- how quickly one protocol starts dominating another over time,
- how RTT changes friendliness,
- and how bandwidth changes the share taken by each protocol.

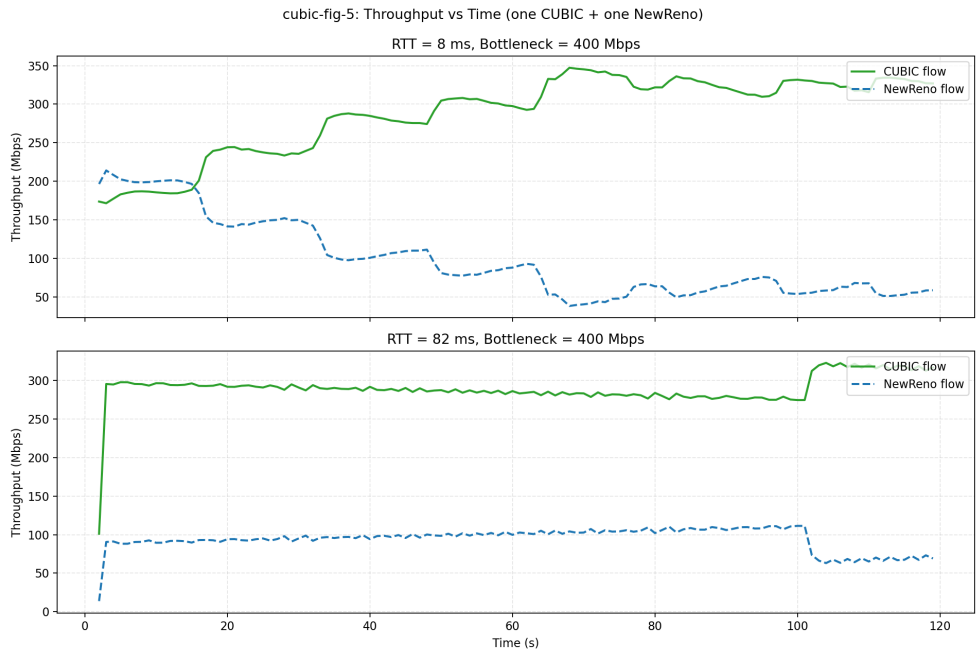


Figure 6: **CUBIC vs. NewReno: Throughput vs. Time at RTT = 8ms and 82ms.** Reproduced from the CUBIC paper experimental setup, this figure shows the time-evolution of throughput when CUBIC competes with NewReno at two different RTT values. At low RTT (8ms), both protocols behave similarly since BDP is small. At high RTT (82ms), CUBIC's cubic window growth function allows it to ramp up significantly faster than NewReno, demonstrating CUBIC's RTT-independence advantage in high-BDP conditions where NewReno's linear growth is insufficient.

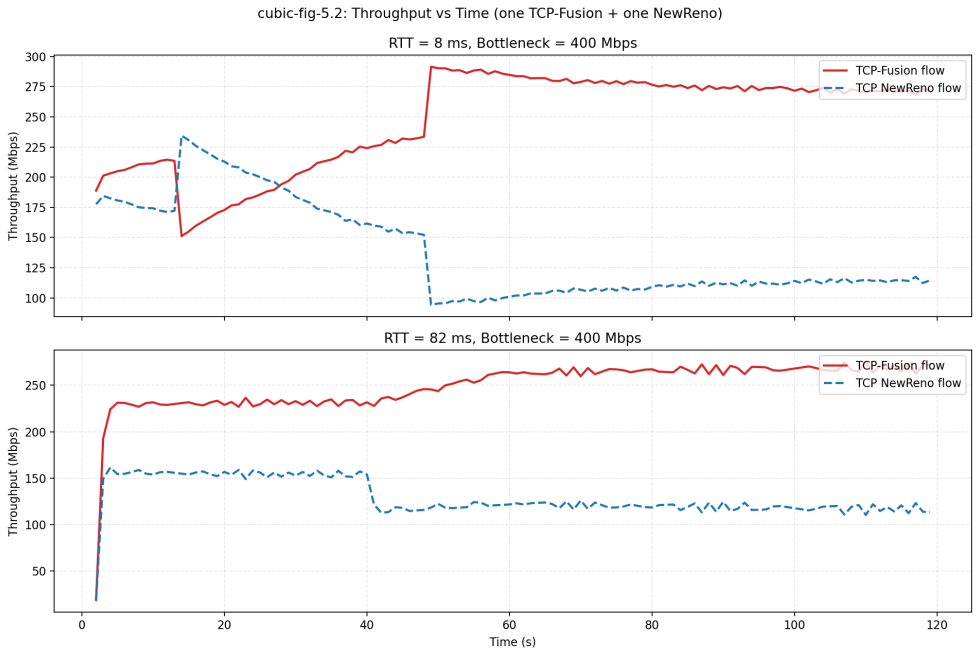


Figure 7: **TCP-Fusion vs. NewReno: Throughput vs. Time at RTT = 8ms and 82ms.** The TCP-Fusion equivalent of Figure 6, directly comparing TCP-Fusion against NewReno under identical RTT conditions. This figure reveals whether TCP-Fusion exhibits time-domain dominance similar to CUBIC or a more balanced coexistence with NewReno. Unlike CUBIC, TCP-Fusion's delay-awareness is expected to prevent runaway window growth, resulting in a more gradual and stable throughput increase that avoids unfairly starving the competing NewReno flow.

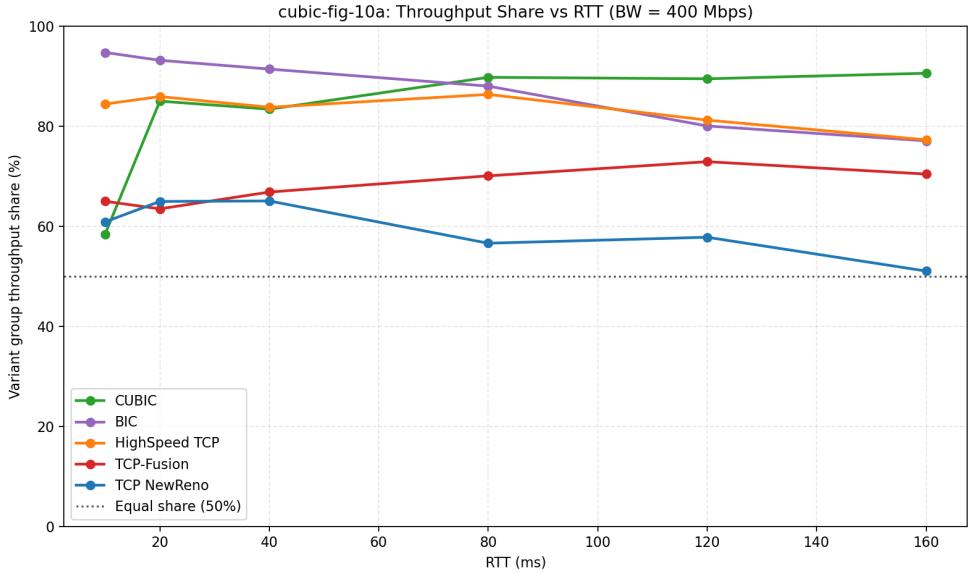


Figure 8: **Throughput Share vs. RTT (Bandwidth Fixed).** This figure plots the throughput *share* — the fraction of total bottleneck bandwidth claimed by each competing protocol — as RTT is varied while keeping bandwidth fixed. Each data point is a run-average share over the full simulation duration, not an instantaneous measurement. A share consistently above 50% indicates aggressiveness at the expense of the competing flow. This figure exposes fairness properties that raw throughput numbers alone cannot reveal, especially under high RTT where window growth rates diverge significantly between protocols.

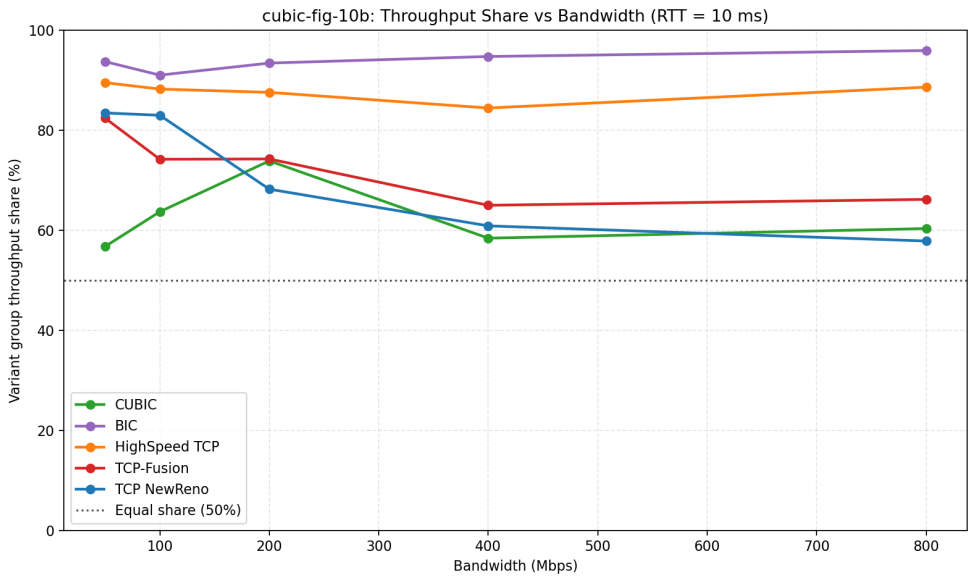


Figure 9: **Throughput Share vs. Bandwidth (RTT = 10ms).** At low RTT (10ms), this figure shows how throughput share between competing protocols changes as bottleneck bandwidth increases. At low RTT the BDP is small and most TCP variants behave similarly, so throughput shares are expected to remain relatively balanced across bandwidth values. Any deviations indicate protocol-specific behaviors triggered by higher capacity availability even under low-latency conditions.

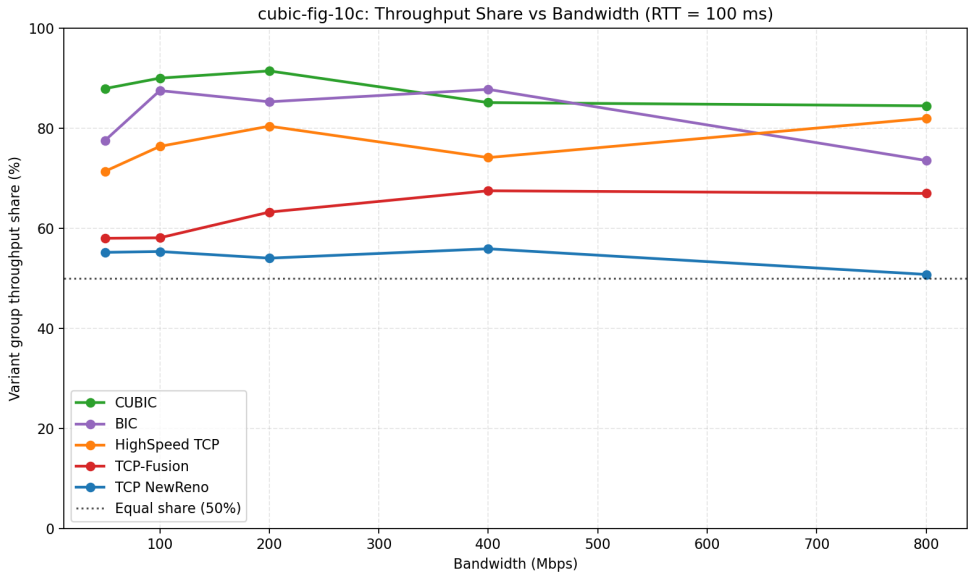


Figure 10: **Throughput Share vs. Bandwidth (RTT = 100ms).** At high RTT (100ms), the BDP is large and differences between TCP variants become pronounced. This figure shows how throughput share evolves as bandwidth increases under high-latency conditions. Protocols designed for high-BDP environments (such as CUBIC and TCP-Fusion) are expected to claim a larger share compared to NewReno, which suffers from slow linear window growth under high RTT. The comparison between this figure and Figure 9 illustrates the RTT-sensitivity of each protocol's aggressiveness.

4 Assigned Mobile Wireless Experiments

4.1 Assignment Specification

The assigned network types are:

- **Wireless 802.11 (mobile)**
- **Wireless 802.15.4 (mobile)**

4.2 Network Topologies

4.2.1 IEEE 802.11 Mobile (Ad-hoc WiFi)

- Standard: 802.11g, Ad-hoc mode (no fixed Access Point)
- Routing Protocol: AODV (Ad-hoc On-Demand Distance Vector)
- Mobility Model: Random Waypoint
- Simulation Area: Square flat grid
- Energy Model: NS-3 WiFi Radio Energy Model

4.2.2 IEEE 802.15.4 Mobile (LR-WPAN / Zigbee)

- Stack: LR-WPAN + 6LoWPAN + IPv6
- Mobility Model: Random Waypoint

- Energy Model: Analytical estimate (Tx/Rx/Idle power terms)
- Max Frame Size: 127 bytes (causes TCP segment fragmentation via 6LoWPAN)

Note: Absolute energy values across WiFi and LR-WPAN are *not* directly comparable due to different energy modeling approaches. This is explicitly documented to avoid unfair cross-technology energy claims.

4.3 Parameters Varied

Since both assigned networks involve **mobile** nodes, the coverage area is fixed and node speed is varied instead. The parameters varied are:

Table 1: Simulation Parameters

Parameter	Values	Baseline
Number of Nodes	20, 40, 60, 80, 100	60
Number of Flows	10, 20, 30, 40, 50	30
Packets per Second	100, 200, 300, 400, 500	300
Node Speed (m/s)	5, 10, 15, 20, 25	15

When varying one parameter, all others are held at their baseline value.

4.4 Metrics Measured

The following metrics are collected and plotted for each parameter variation:

1. **Network Throughput** (Mbps)
2. **End-to-End Delay** (ms)
3. **Packet Delivery Ratio (PDR)** $= \frac{\text{total packets received}}{\text{total packets sent}}$
4. **Packet Drop Ratio** $= \frac{\text{total packets dropped}}{\text{total packets sent}}$
5. **Energy Consumption** (Joules) — applicable for all wireless nodes

4.5 Implementation Files

The assigned experiments are located under `scratch/assigned_experiments/`:

```
tcp-fusion-wifi-mobile.cc        # 802.11 mobile simulation script
tcp-fusion-lrwpan-mobile.cc      # 802.15.4 mobile simulation
   script
run_mobile_experiments.py        # Automation script for all runs
plot_mobile_metrics.py           # Plotting pipeline
config_mobile.json               # Experiment configuration
README.md                        # Instructions
REPORT_TEMPLATE.md               # Report template
results/mobile_raw_results.csv   # Generated raw data
results/plots/                   # Generated plots
```

5 Results and Observations

5.1 Baseline Comparison

At baseline parameters (nNodes = 60, nFlows = 30, pktPerSec = 300, speed = 15 m/s):

Table 2: Baseline Performance: WiFi vs. LR-WPAN (TCP-Fusion)

Network	Throughput	Delay	PDR	Drop Ratio	Energy
WiFi 802.11 (mobile)	2.794 Mbps	27.50 ms	0.8928	0.1072	2995.59 J
LR-WPAN 802.15.4 (mobile)	0.0905 Mbps	291.23 ms	0.7011	0.2989	11.36 J

WiFi delivers substantially higher throughput and much lower delay compared to LR-WPAN. LR-WPAN's energy consumption is orders of magnitude lower, consistent with its low-power sensor network design.

5.2 Parameter Variation Trends

5.2.1 A) Varying Number of Nodes

- **WiFi:** Throughput falls significantly as nodes increase due to channel contention and route instability. Delay and drop ratio increase.
- **LR-WPAN:** Throughput remains around ∼0.09 Mbps regardless of node count; delay is persistently high, and reliability is moderate-to-low.

5.2.2 B) Varying Number of Flows

- **WiFi:** Total throughput increases with more flows up to channel saturation, but delay and drops also rise.
- **LR-WPAN:** Throughput saturates early; additional flows primarily increase delay and drop ratio, significantly reducing PDR.

5.2.3 C) Varying Packet Rate

- **WiFi:** Throughput grows with offered load and PDR remains comparatively stable.
- **LR-WPAN:** Only marginal throughput gains; high delay persists, indicating a bottlenecked MAC/PHY layer incapable of handling high offered loads.

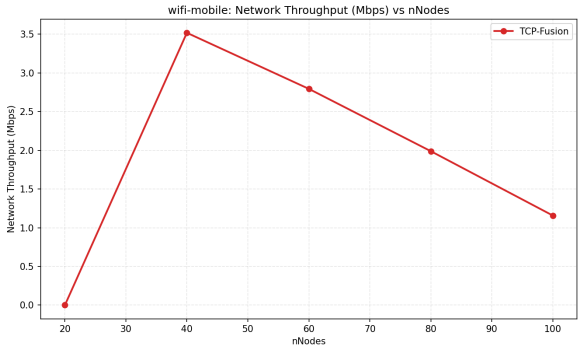
5.2.4 D) Varying Node Speed

- **WiFi:** Higher speed degrades throughput and PDR while increasing drops, due to more frequent topology changes and AODV route breaks.
- **LR-WPAN:** Speed impact is weaker than load impact; overall throughput stays in a narrow low band regardless of speed.

5.3 Assigned Experiment Plots

5.3.1 Throughput

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

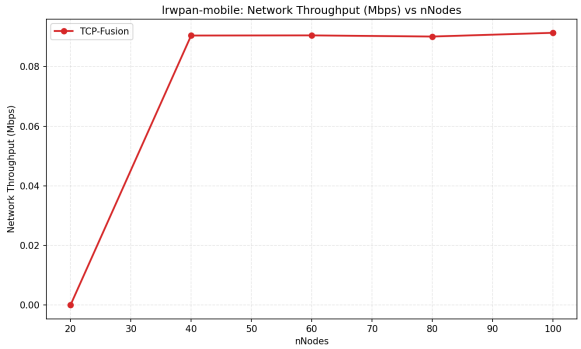
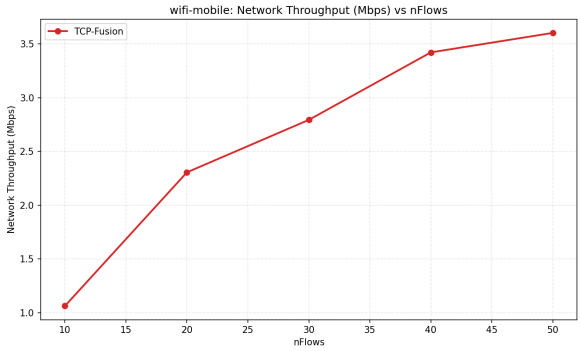


Figure 11: Throughput vs Number of Nodes

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

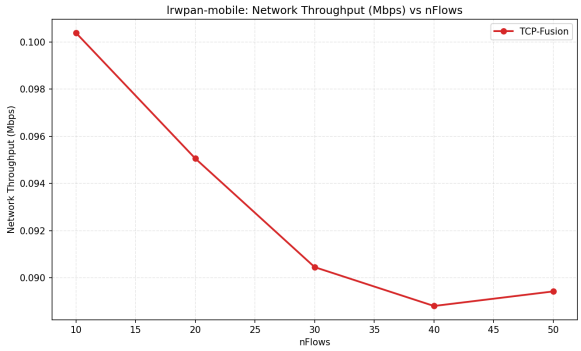
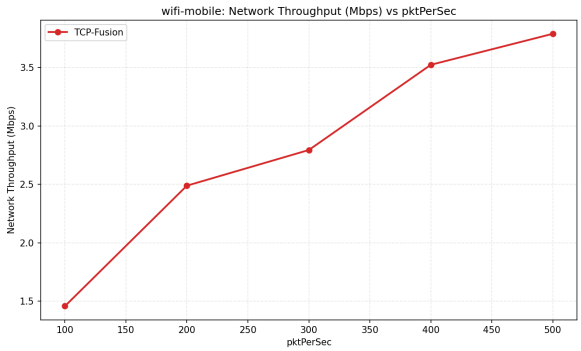


Figure 12: Throughput vs Number of Flows

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

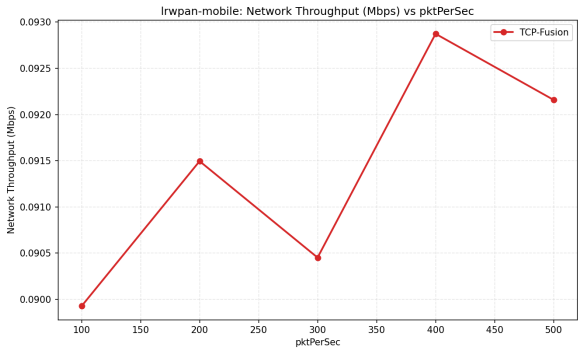
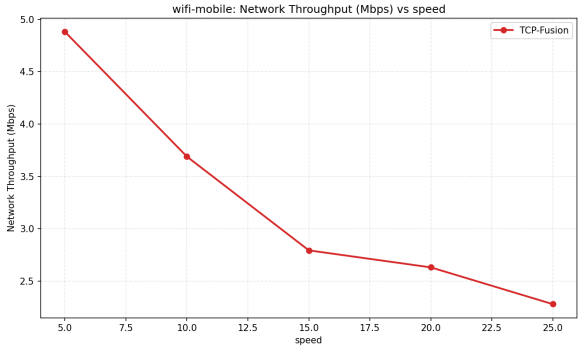


Figure 13: Throughput vs Packet Rate

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

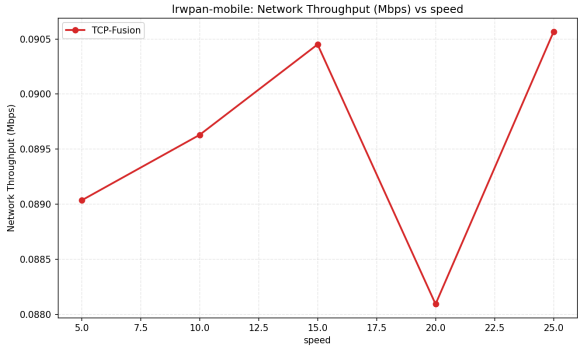
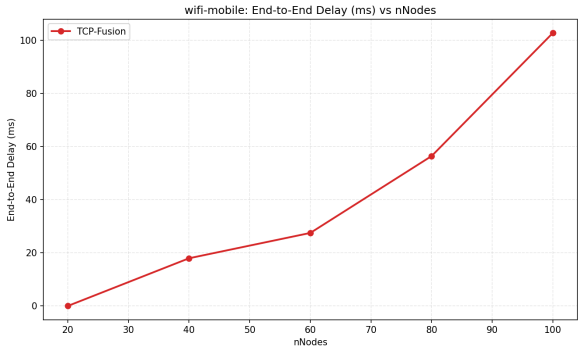


Figure 14: Throughput vs Node Speed

5.3.2 End-to-End Delay

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

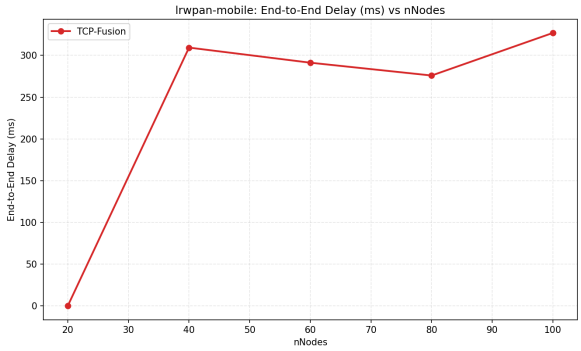
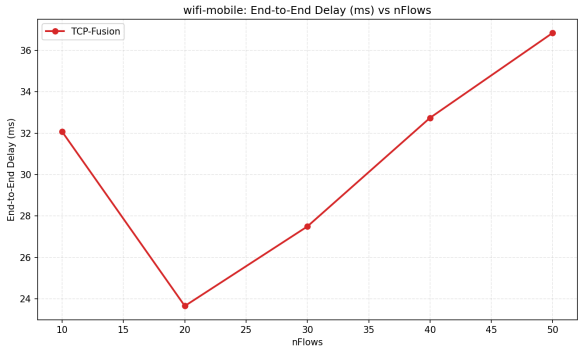


Figure 15: End-to-End Delay vs Number of Nodes

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

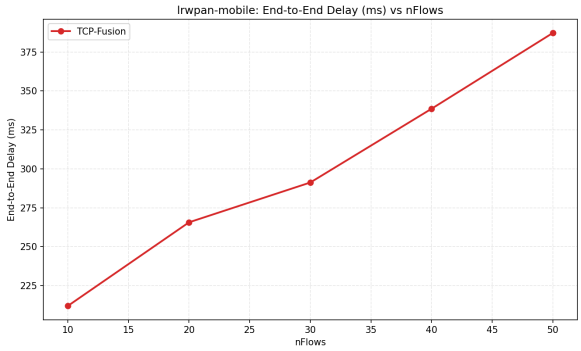
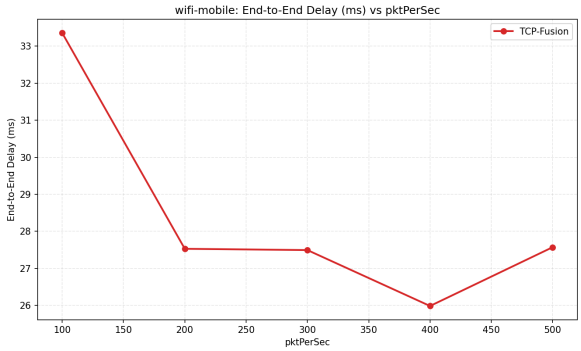


Figure 16: End-to-End Delay vs Number of Flows

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

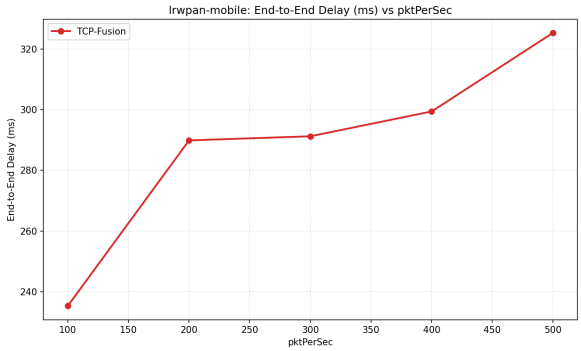
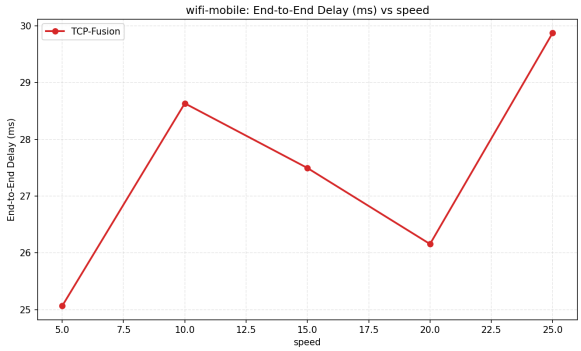


Figure 17: End-to-End Delay vs Packet Rate

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

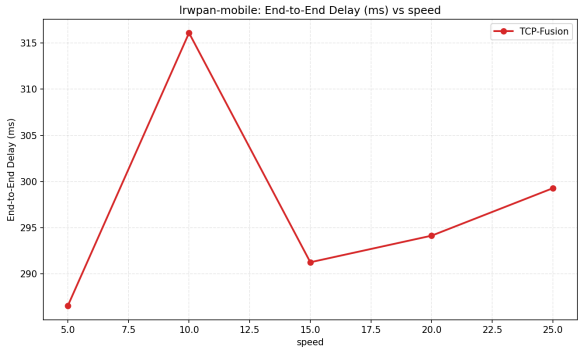
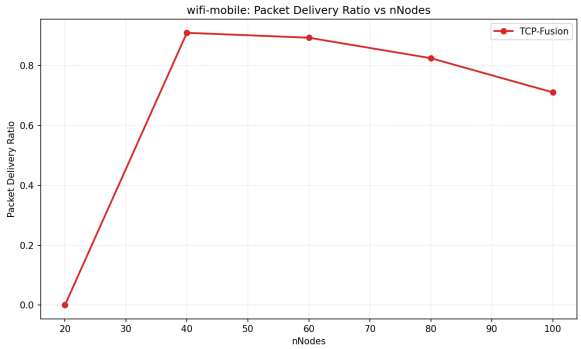


Figure 18: End-to-End Delay vs Node Speed

5.3.3 Packet Delivery Ratio (PDR)

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

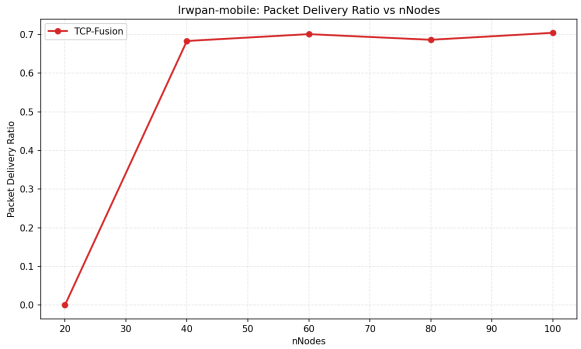
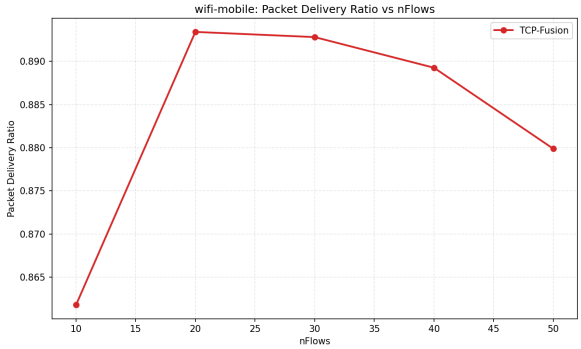


Figure 19: PDR vs Number of Nodes

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

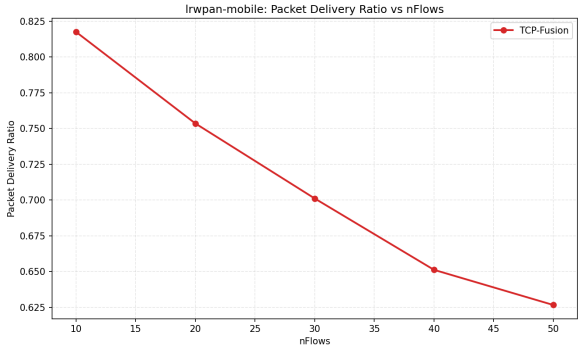
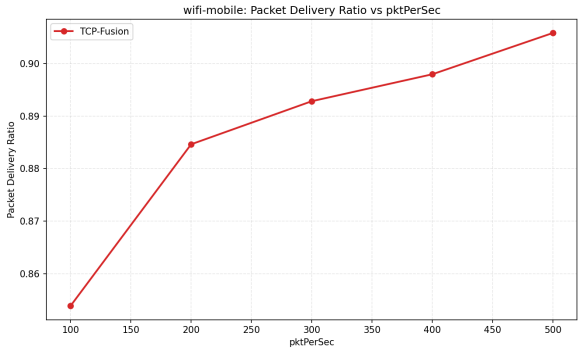


Figure 20: PDR vs Number of Flows

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

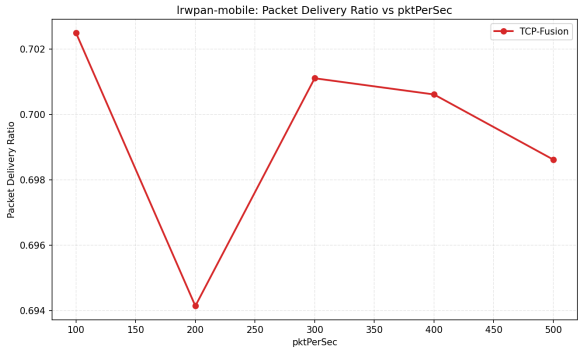
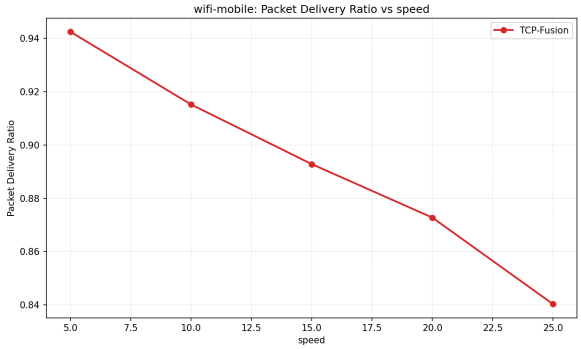


Figure 21: PDR vs Packet Rate

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

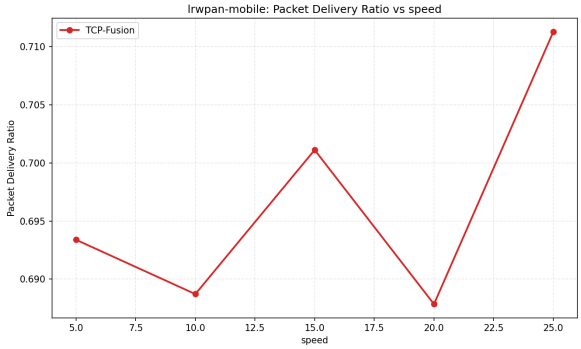
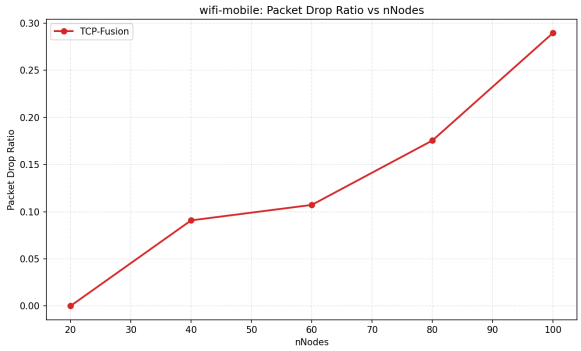


Figure 22: PDR vs Node Speed

5.3.4 Packet Drop Ratio

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

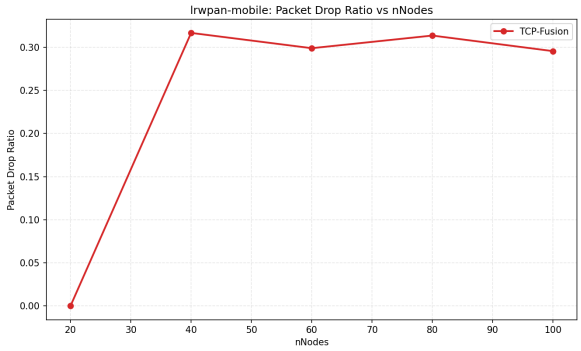
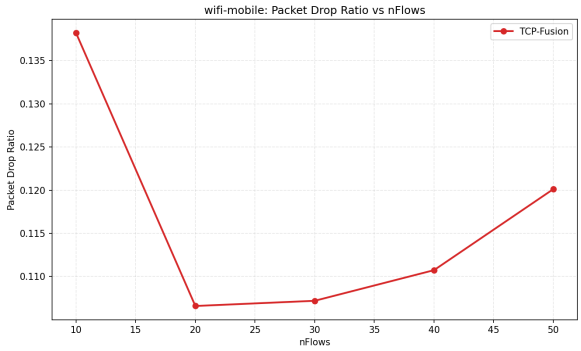


Figure 23: Drop Ratio vs Number of Nodes

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

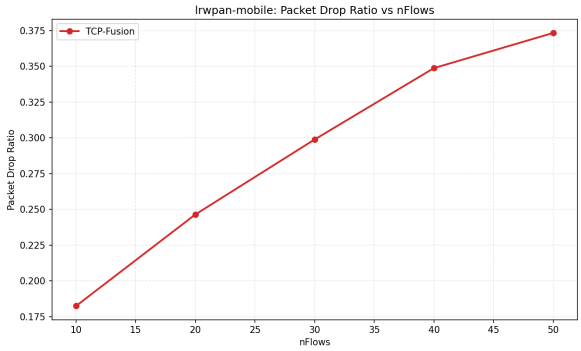
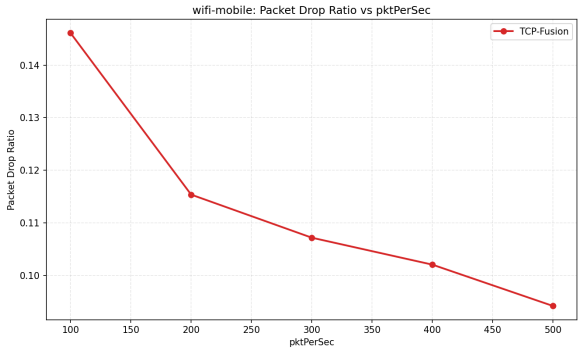


Figure 24: Drop Ratio vs Number of Flows

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

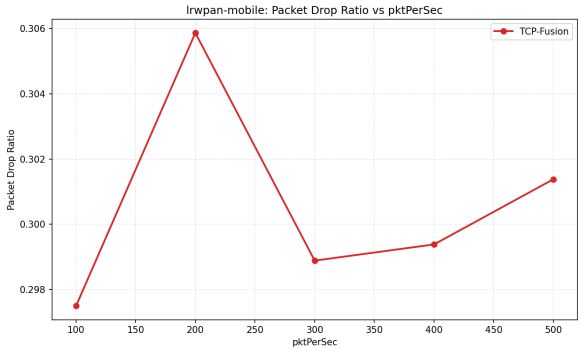
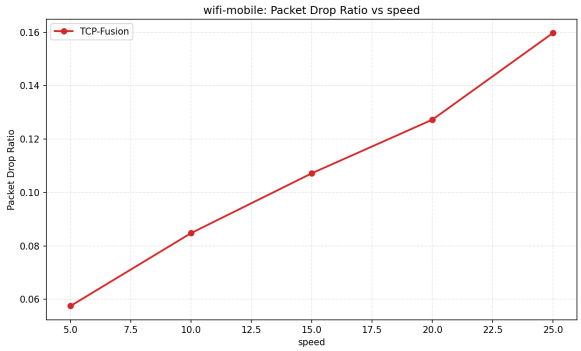


Figure 25: Drop Ratio vs Packet Rate

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

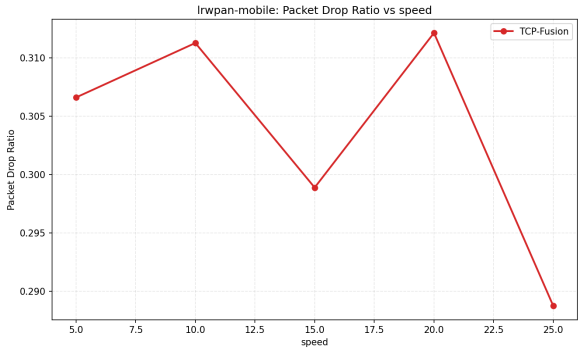
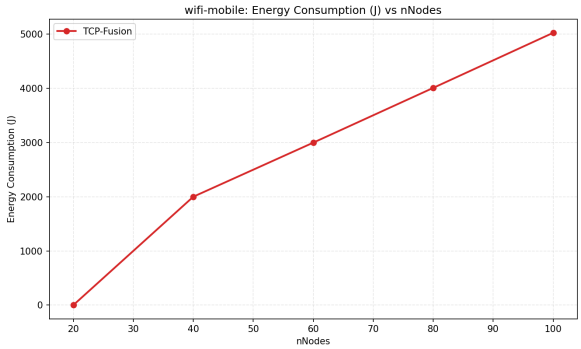


Figure 26: Drop Ratio vs Node Speed

5.3.5 Energy Consumption

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

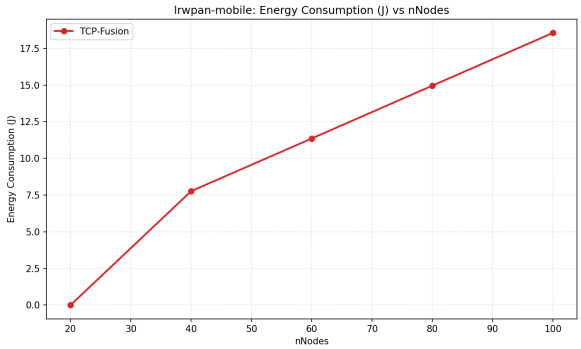
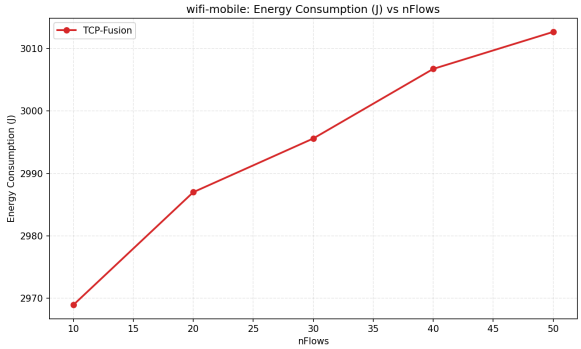


Figure 27: Energy Consumption vs Number of Nodes

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

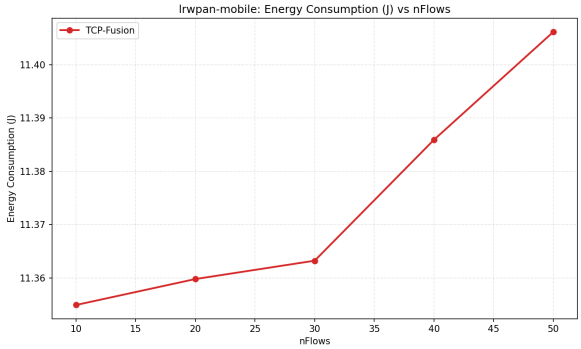
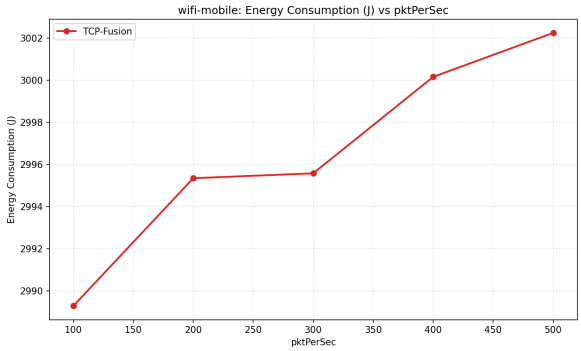


Figure 28: Energy Consumption vs Number of Flows

WiFi 802.11 (mobile)



LR-WPAN 802.15.4 (mobile)

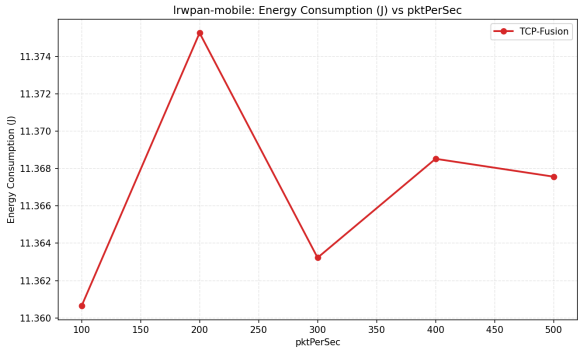


Figure 29: Energy Consumption vs Packet Rate

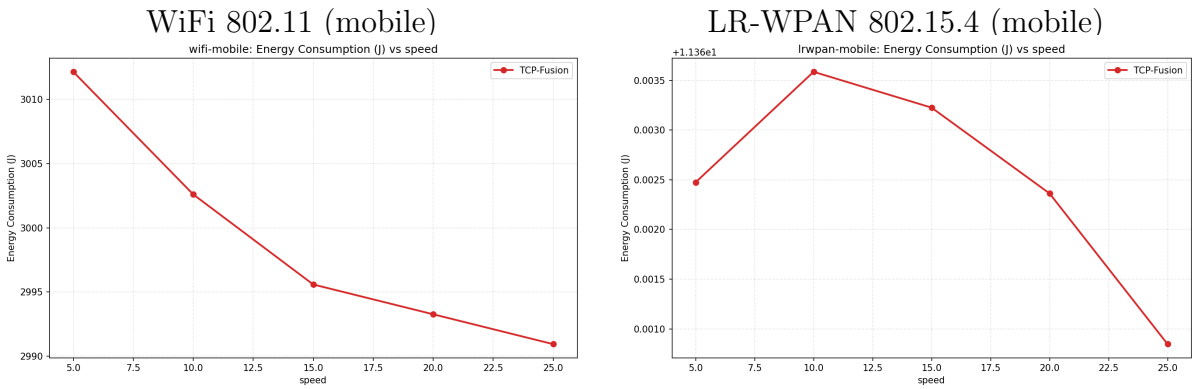


Figure 30: Energy Consumption vs Node Speed

6 Discussion

6.1 Why WiFi Significantly Outperforms LR-WPAN

The results are consistent with the fundamental design differences between the two technologies. IEEE 802.11g supports data rates up to 54 Mbps with a typical range of 50–150m, whereas IEEE 802.15.4 is limited to 250 Kbps by design, targeting low-power sensor applications. Running TCP (a protocol designed for reliable, high-throughput data transfer) over 802.15.4 introduces structural mismatches:

- TCP segments exceed the 127-byte 802.15.4 frame limit, requiring fragmentation via 6LoWPAN. Any lost fragment causes the entire TCP segment to be retransmitted.
- High delay in LR-WPAN causes TCP's retransmission timers to fire frequently, further degrading throughput.
- The low channel capacity saturates quickly under even modest flow counts.

6.1.1 Clarification of Observed Trends

Some trends may look counter-intuitive at first glance, but they are consistent with how the metrics are defined and with the operating region of each technology:

- **Throughput vs number of flows (WiFi):** the plotted value is *aggregate network throughput*, not per-flow throughput. As flow count increases, total offered load increases and the channel is utilized more fully, so aggregate throughput can rise up to saturation.
- **Throughput vs number of flows (LR-WPAN):** LR-WPAN reaches saturation very early due to low PHY capacity. Therefore, adding flows does not increase useful throughput much; performance mostly shifts toward higher contention, delay, and drop ratio.
- **Delay vs packet rate:** in LR-WPAN, delay increases with packet rate because queues build quickly at the MAC/PHY bottleneck. In WiFi, delay can remain stable or even slightly decrease over part of the load range due to improved airtime utilization before hard saturation.
- **Throughput vs node speed:** in WiFi, higher speed causes more frequent route

changes (AODV repairs), reducing throughput. In LR-WPAN, throughput is already near a low-capacity ceiling, so speed mostly causes small fluctuations around that ceiling rather than strong monotonic changes.

Hence, the apparent "increase" in some LR-WPAN points is best interpreted as variance around a saturated operating point, not as genuine scalability gain.

6.2 How TCP-Fusion Helps in Wireless Environments

Pure loss-driven TCP Reno incorrectly interprets wireless link losses (due to signal fading, node mobility, and route breaks) as congestion signals and unnecessarily reduces the congestion window. TCP-Fusion's Vegas-like delay sensing allows it to distinguish between:

- **Congestion-induced loss** → reduce window (correct response)
- **Mobility/signal loss** → maintain or recover window faster (avoids unnecessary throughput degradation)

This makes TCP-Fusion particularly well-suited for mobile wireless environments where non-congestion losses are frequent.

7 Summary of Findings

In this work, TCP-Fusion was fully implemented in NS-3 as a new congestion control variant that combines Reno compatibility, Vegas-style delay awareness, and Westwood-style bandwidth-guided growth. The implementation was validated through single-flow, two-flow, cwnd-time, and three-flow fairness experiments. From these experiments, we observed that TCP-Fusion generally improves utilization under loss and maintains better stability than purely loss-driven behavior, while still keeping reasonable coexistence with Reno compared to more aggressive high-speed variants.

The extended CUBIC-style experiments helped us understand what the initial loss-based graphs could not show clearly: time-domain dominance and throughput-share behavior under different RTT and bandwidth conditions. Finally, in the assigned mobile wireless scenarios, the results showed a clear technology-level contrast: 802.11 mobile delivered much higher throughput and lower delay, while 802.15.4 mobile remained low-power but quickly capacity-limited under TCP load. Overall, the combined results suggest that TCP-Fusion meets its intended direction—better high-speed utilization with improved practical friendliness—while also highlighting that network technology constraints (especially in LR-WPAN) can dominate transport-level gains.